

pso_rc_tuner.v — PSO-Based RC Filter Auto-Tuner (Design Code)

Verilog HDL design module | NIT Warangal EC1211 Digital Circuits Design Lab Mini Project

```
`timescale 1ns/1ps
// PSO-based auto-tuner for RC cutoff frequency ( $f_c = 1/(2\pi RC)$ ).
module pso_rc_tuner;
    // User parameters (unchanged functionality)
    parameter integer N_PARTICLES = 12;
    parameter integer N_ITER = 25;
    parameter real F_DESIRED = 2500.0;          // Hz
    parameter real R_MIN = 1.0e3;              // Ohms
    parameter real R_MAX = 1.0e6;              // Ohms
    parameter real C_MIN = 100e-12;            // Farads
    parameter real C_MAX = 1.0e-6;             // Farads
    parameter real C1 = 1.6;                   // cognitive
    parameter real C2 = 1.6;                   // social

    // Inertia schedule
    parameter real W_START = 0.9;
    parameter real W_END = 0.5;

    // Exploration/constraints (unchanged)
    parameter real VCLAMP_R_F = 0.10;          // max |v_R| as fraction of R-range
    parameter real VCLAMP_C_F = 0.10;          // max |v_C| as fraction of C-range
    parameter real EDGE_MARGIN_F = 0.02;        // 2% near edges
    parameter real EDGE_PENALTY_PCT = 0.10;     // up to +0.10% error when near edges
    parameter integer MAX_WORSE_STEPS = 6;
    parameter integer ENABLE_GREEDY_REPAIR = 1;

    localparam real TWO_PI = 6.283185307179586;

    // Particle state (64-bit "real" via bits)
    reg [63:0] pos_R [0:N_PARTICLES-1];
    reg [63:0] pos_C [0:N_PARTICLES-1];
    reg [63:0] vel_R [0:N_PARTICLES-1];
    reg [63:0] vel_C [0:N_PARTICLES-1];
    reg [63:0] pbest_R [0:N_PARTICLES-1];
    reg [63:0] pbest_C [0:N_PARTICLES-1];
    reg [63:0] pbest_err [0:N_PARTICLES-1];
    integer off_path_cnt [0:N_PARTICLES-1];
    integer active [0:N_PARTICLES-1];

    // Global best
    reg [63:0] gbest_R_bits, gbest_C_bits, gbest_err_bits;
    integer gbest_idx;

    // Loop indices and temps
    integer i, j, it;
    integer active_count, last_active_idx;

    // Temps for math
    reg [63:0] r1_b, r2_b;
    reg [63:0] new_vr_b, new_vc_b;
    reg [63:0] new_pr_b, new_pc_b;
    reg [63:0] new_vr_adj_b, new_vc_adj_b;
    reg [63:0] fc_b, err_b;
    reg [63:0] fc_prev_b, err_prev_b;
    real prev_best_err;

    // Greedy repair temps
    reg [63:0] fc_tmp_b, err_orig_b, err_rep_b;
    reg [63:0] rep_R_b, rep_C_b;

    // Iteration logging helpers
    integer disabled_this_iter [0:N_PARTICLES-1];
```

```

integer cnt_before, act_before;
real gbest_err_before;
integer gbest_idx_before;

// Loop control temps
integer stop_now;
integer first;

// Final report
real Rf, Cf, fcf;

// Shared temp
reg [63:0] u_b;
real u, vr0, vc0;

// Gbest print temps
real gR, gC, gfc, gerr;

// Helpers: bit<math>\rightarrow</math>real
function real bits_to_real;
    input [63:0] b; begin bits_to_real = $bitstoreal(b); end
endfunction

function [63:0] real_to_bits;
    input real x; begin real_to_bits = $realtobits(x); end
endfunction

// RNG: uniform [0,1)
task rng01_bits;
    output [63:0] ubo;
    real ur;
    begin
        ur = $urandom / 4294967296.0; // [0,1)
        if (ur >= 1.0) ur = 0.999999999;
        ubo = real_to_bits(ur);
    end
endtask

// RNG: uniform [lo,hi]
task rng_range_bits;
    input real lo, hi;
    output [63:0] out_b;
    reg [63:0] ub;
    real ur, v;
    begin
        rng01_bits(ub);
        ur = bits_to_real(ub);
        v = lo + ur * (hi - lo);
        out_b = real_to_bits(v);
    end
endtask

// Velocity update (dynamic inertia via it/N_ITER)
task velocity_update_bits;
    input [63:0] pos_R_b, pos_C_b, vel_R_b, vel_C_b, pbest_R_b, pbest_C_b, gbest_R_b, gbest_C_b, r1_bi, r2_bi;
    output [63:0] vel_R_out_b, vel_C_out_b;
    real pr, pc, vr, vc, pbr, pbc, gbr, gbc, r1, r2, nvr, nvc;
    real iter_frac, W_CUR;
    begin
        pr = bits_to_real(pos_R_b);
        pc = bits_to_real(pos_C_b);
        vr = bits_to_real(vel_R_b);
        vc = bits_to_real(vel_C_b);
        pbr = bits_to_real(pbest_R_b);
        pbc = bits_to_real(pbest_C_b);
        gbr = bits_to_real(gbest_R_b);
        gbc = bits_to_real(gbest_C_b);

```

```

    r1 = bits_to_real(r1_bi);
    r2 = bits_to_real(r2_bi);

    if (N_ITER > 1) begin
        iter_frac = it / (N_ITER - 1.0);
        W_CUR = W_START - (W_START - W_END) * iter_frac;
    end else begin
        W_CUR = W_END;
    end

    nvr = W_CUR*vr + C1*r1*(pbr - pr) + C2*r2*(gbr - pr);
    nvc = W_CUR*vc + C1*r1*(pbc - pc) + C2*r2*(gbc - pc);

    vel_R_out_b = real_to_bits(nvr);
    vel_C_out_b = real_to_bits(nvc);
end
endtask

// Fitness: fc and percent error; includes small edge penalty
task fitness_bits;
    input [63:0] R_b, C_b;
    output [63:0] fc_bo, err_bo;
    real R, C, fc, de, perr;
    real r_norm, c_norm, pen;
    begin
        R = bits_to_real(R_b);
        C = bits_to_real(C_b);
        fc = 1.0 / (TWO_PI * R * C);
        de = (fc > F_DESIRE) ? (fc - F_DESIRE) : (F_DESIRE - fc);
        perr = (F_DESIRE > 0.0) ? (de / F_DESIRE * 100.0) : 1.0e300;

        // Edge penalty
        r_norm = (R - R_MIN) / (R_MAX - R_MIN);
        c_norm = (C - C_MIN) / (C_MAX - C_MIN);
        pen = 0.0;
        if (EDGE_MARGIN_F > 0.0) begin
            if (r_norm < EDGE_MARGIN_F)
                pen = pen + EDGE_PENALTY_PCT * (EDGE_MARGIN_F - r_norm) / EDGE_MARGIN_F;
            else if (r_norm > (1.0 - EDGE_MARGIN_F))
                pen = pen + EDGE_PENALTY_PCT * (r_norm - (1.0 - EDGE_MARGIN_F)) / EDGE_MARGIN_F;

            if (c_norm < EDGE_MARGIN_F)
                pen = pen + 0.5*EDGE_PENALTY_PCT * (EDGE_MARGIN_F - c_norm) / EDGE_MARGIN_F;
            else if (c_norm > (1.0 - EDGE_MARGIN_F))
                pen = pen + 0.5*EDGE_PENALTY_PCT * (c_norm - (1.0 - EDGE_MARGIN_F)) / EDGE_MARGIN_F;
        end
        perr = perr + pen;

        fc_bo = real_to_bits(fc);
        err_bo = real_to_bits(perr);
    end
endtask

// Global best from pbest across active particles
task update_gbest;
    integer jj;
    real best_e, e;
    integer found;
    begin
        best_e = 1.0e300;
        found = 0;
        for (jj = 0; jj < N_PARTICLES; jj = jj + 1) begin
            if (active[jj]) begin
                e = bits_to_real(pbest_err[jj]);
                if (!found || (e < best_e)) begin
                    best_e = e;
                    gbest_R_bits = pbest_R[jj];
                end
            end
        end
    end
endtask

```

```

        gbest_C_bits = pbest_C[jj];
        gbest_idx = jj;
        found = 1;
    end
end
end
if (found) gbest_err_bits = real_to_bits(best_e);
end
endtask

// DPSH: deactivate after MAX_WORSE_STEPS consecutive worse steps
task dpsh_update;
    input [63:0] curr_err_b, pbest_err_prev_b;
    inout integer cnt, act;
    real ce, pe;
    begin
        ce = bits_to_real(curr_err_b);
        pe = bits_to_real(pbest_err_prev_b);
        if (ce > pe) cnt = cnt + 1; else cnt = 0;
        if (cnt >= MAX_WORSE_STEPS) act = 0;
    end
endtask

// Position update with reflecting bounds and velocity clamp
task position_update_bits;
    input [63:0] pos_R_b, pos_C_b, vel_R_b, vel_C_b;
    output [63:0] pos_R_out_b, pos_C_out_b, vel_R_out_b, vel_C_out_b;
    real pr, pc, vr, vc, nr, nc;
    real rspan, cspan, vmax_r, vmax_c;
    integer reflect_iter;
    begin
        pr = bits_to_real(pos_R_b);
        pc = bits_to_real(pos_C_b);
        vr = bits_to_real(vel_R_b);
        vc = bits_to_real(vel_C_b);

        rspan = (R_MAX - R_MIN);
        cspan = (C_MAX - C_MIN);
        vmax_r = VCLAMP_R_F * rspan;
        vmax_c = VCLAMP_C_F * cspan;

        if (vr > vmax_r) vr = vmax_r; else if (vr < -vmax_r) vr = -vmax_r;
        if (vc > vmax_c) vc = vmax_c; else if (vc < -vmax_c) vc = -vmax_c;

        nr = pr + vr;
        nc = pc + vc;

        for (reflect_iter = 0; reflect_iter < 4; reflect_iter = reflect_iter + 1) begin
            if (nr < R_MIN) begin
                nr = 2.0*R_MIN - nr; vr = -0.5*vr;
            end else if (nr > R_MAX) begin
                nr = 2.0*R_MAX - nr; vr = -0.5*vr;
            end
            if (nc < C_MIN) begin
                nc = 2.0*C_MIN - nc; vc = -0.5*vc;
            end else if (nc > C_MAX) begin
                nc = 2.0*C_MAX - nc; vc = -0.5*vc;
            end
            if ((nr >= R_MIN && nr <= R_MAX) && (nc >= C_MIN && nc <= C_MAX))
                reflect_iter = 4; // exit
        end
    end

    if (nr < R_MIN) nr = R_MIN; if (nr > R_MAX) nr = R_MAX;
    if (nc < C_MIN) nc = C_MIN; if (nc > C_MAX) nc = C_MAX;

    pos_R_out_b = real_to_bits(nr);

```

```

        pos_C_out_b = real_to_bits(nc);
        vel_R_out_b = real_to_bits(vr);
        vel_C_out_b = real_to_bits(vc);
    end
endtask

// Greedy repair toward target fc for current R,C
task repair_choose_best;
    input [63:0] Rin_b, Cin_b;
    output [63:0] Rout_b, Cout_b;
    real R_in, C_in, R1, C1loc, R2, C2loc, Cstar, Rstar;
    reg [63:0] fc_b1, err_b1, fc_b2, err_b2;
    real e1, e2;
    begin
        R_in = bits_to_real(Rin_b);
        C_in = bits_to_real(Cin_b);

        // Candidate 1: fix R, solve C*
        Cstar = 1.0 / (TWO_PI * R_in * F_DESIRED);
        if (Cstar < C_MIN) C1loc = C_MIN; else if (Cstar > C_MAX) C1loc = C_MAX; else C1loc = Cstar;
        R1 = R_in;

        // Candidate 2: fix C, solve R*
        Rstar = 1.0 / (TWO_PI * C_in * F_DESIRED);
        if (Rstar < R_MIN) R2 = R_MIN; else if (Rstar > R_MAX) R2 = R_MAX; else R2 = Rstar;
        C2loc = C_in;

        // Evaluate both
        fitness_bits(real_to_bits(R1), real_to_bits(C1loc), fc_b1, err_b1);
        fitness_bits(real_to_bits(R2), real_to_bits(C2loc), fc_b2, err_b2);
        e1 = bits_to_real(err_b1);
        e2 = bits_to_real(err_b2);

        if (e1 <= e2) begin
            Rout_b = real_to_bits(R1);
            Cout_b = real_to_bits(C1loc);
        end else begin
            Rout_b = real_to_bits(R2);
            Cout_b = real_to_bits(C2loc);
        end
    end
end
endtask

// Main
initial begin
    // Title and inputs
    $display("&quot;-----&quot;");
    $display("&quot;Particle Swarm Optimization based RC Tuner using Dynamic Particle Size Handling&quot;");
    $display("&quot;Inputs: F_DESIRED=%.6f Hz, R_MIN=%.3f, R_MAX=%.3f, C_MIN=%.3e, C_MAX=%.3e&quot;;, F_DESIRED");
    $display("&quot;Particles: %0d, Iterations: %0d&quot;;, N_PARTICLES, N_ITER);
    $display("&quot;-----&quot;");

    // Initialize particles and print initial states
    for (i = 0; i < N_PARTICLES; i = i + 1) begin
        rng_range_bits(R_MIN, R_MAX, pos_R[i]);
        rng_range_bits(C_MIN, C_MAX, pos_C[i]);

        // small random initial velocity within ±5% of range
        rng01_bits(u_b); u = 2.0*bits_to_real(u_b) - 1.0; vr0 = 0.05*(R_MAX-R_MIN)*u;
        rng01_bits(u_b); u = 2.0*bits_to_real(u_b) - 1.0; vc0 = 0.05*(C_MAX-C_MIN)*u;

        vel_R[i] = real_to_bits(vr0);
        vel_C[i] = real_to_bits(vc0);

        pbest_R[i] = pos_R[i];
        pbest_C[i] = pos_C[i];
        fitness_bits(pos_R[i], pos_C[i], fc_b, err_b);
    end
end

```

```

pbest_err[i] = err_b;
off_path_cnt[i] = 0;
active[i] = 1;

// Initial state line
$display("&P%0d: R=%.3f C=%.3e fc=%.6f err=%.6f%&quot;;",
i, bits_to_real(pos_R[i]), bits_to_real(pos_C[i]),
bits_to_real(fc_b), bits_to_real(err_b));
end

// Initial global best
update_gbest();
$display("&quot;----&quot;);

// Iterations
for (it = 0; it &lt; N_ITER; it = it + 1) begin
// clear disabled list for this iteration
for (j = 0; j &lt; N_PARTICLES; j = j + 1) disabled_this_iter[j] = 0;
active_count = 0;
last_active_idx = -1;

// update all particles
for (i = 0; i &lt; N_PARTICLES; i = i + 1) begin
if (active[i]) begin
// Random coefficients
rng01_bits(r1_b);
rng01_bits(r2_b);

// Velocity and position
velocity_update_bits(pos_R[i], pos_C[i], vel_R[i], vel_C[i], pbest_R[i], pbest_C[i], gbest_R_
position_update_bits(pos_R[i], pos_C[i], new_vr_b, new_vc_b, new_pr_b, new_pc_b, new_vr_adj_b

// Greedy repair (optional)
if (ENABLE_GREEDY_REPAIR) begin
fitness_bits(new_pr_b, new_pc_b, fc_tmp_b, err_orig_b);
repair_choose_best(new_pr_b, new_pc_b, rep_R_b, rep_C_b);
fitness_bits(rep_R_b, rep_C_b, fc_tmp_b, err_rep_b);
if (bits_to_real(err_rep_b) &lt; bits_to_real(err_orig_b)) begin
new_pr_b = rep_R_b;
new_pc_b = rep_C_b;
end
end

vel_R[i] = new_vr_adj_b; vel_C[i] = new_vc_adj_b;
pos_R[i] = new_pr_b; pos_C[i] = new_pc_b;

// Fitness and DPSH/pbest
fitness_bits(pos_R[i], pos_C[i], fc_b, err_b);

// DPSH around update
cnt_before = off_path_cnt[i];
act_before = active[i];
prev_best_err = bits_to_real(pbest_err[i]);
dpsht_update(err_b, pbest_err[i], off_path_cnt[i], active[i]);

if (active[i] && (bits_to_real(err_b) &lt; prev_best_err)) begin
pbest_err[i] = err_b;
pbest_R[i] = pos_R[i];
pbest_C[i] = pos_C[i];
end

if (act_before && !active[i]) begin
disabled_this_iter[i] = 1;
end

if (active[i]) begin
active_count = active_count + 1;

```

```

        last_active_idx = i;
    end
end
end

// Decide GBEST for this iteration and whether to stop
stop_now = 0;
if (active_count == 1) begin
    fitness_bits(pos_R[last_active_idx], pos_C[last_active_idx], fc_b, err_b);
    gbest_R_bits = pos_R[last_active_idx];
    gbest_C_bits = pos_C[last_active_idx];
    gbest_err_bits = err_b;
    gbest_idx = last_active_idx;
    stop_now = 1;
end else begin
    update_gbest();
    if (active_count == 0) stop_now = 1;
end

// Print iteration summary: GBEST and disabled list
gR = bits_to_real(gbest_R_bits);
gC = bits_to_real(gbest_C_bits);
gfc = 1.0 / (TWO_PI * gR * gC);
gerr = bits_to_real(gbest_err_bits);

$write(&quot;Iter %0d: GBEST=P%0d R=%.3f C=%.3e fc=%.6f err=%.6f%% | Disabled:&quot;;, it, gbest_idx,
first = 1;
for (j = 0; j &lt; N_PARTICLES; j = j + 1) begin
    if (disabled_this_iter[j]) begin
        if (!first) $write(&quot;;&quot;);
        $write(&quot; P%0d&quot;;, j);
        first = 0;
    end
end
if (first) $write(&quot; none&quot;);
$display(&quot;&quot;);

if (stop_now) begin
    it = N_ITER; // exit after this iteration
end
end

// Final report
$display(&quot;-----&quot;);
Rf = bits_to_real(gbest_R_bits);
Cf = bits_to_real(gbest_C_bits);
fcf = 1.0 / (TWO_PI * Rf * Cf);
$display(&quot;Final: GBEST=P%0d R=%.3f C=%.3e fc=%.6f Err=%.6e%%&quot;;, gbest_idx, Rf, Cf, fcf, bits_to_
$display(&quot;-----&quot;);

$finish;
end
endmodule

```